

AI-Based Document Data Extraction System – Requirement Specification

1. Project Overview

The objective of this project is to build a web-based application capable of extracting structured data from uploaded documents such as:

- Scanned images
- Handwritten forms
- Printed forms
- PDF documents
- Mixed handwritten + printed content

The system should process uploaded documents and return extracted information in JSON format.

Example Output:

```
{
  "name": "Riya Sharma",
  "city": "Manali",
  "state": "Himachal Pradesh",
  "occupation": "Tour Guide"
}
```

The solution should support extraction of both:

- Printed text
- Handwritten text

The uploaded sample form should be considered as one of the reference document types.

2. Functional Requirements

2.1 Frontend Requirements (React)

The frontend application should be developed using React.

Features

1. Upload Interface

2. Users should be able to upload:
 - Images (JPG, PNG, JPEG)
 - PDF documents
 3. Drag-and-drop upload support
 4. Multiple file upload support (optional)
 5. Preview Functionality
 6. Display uploaded image/PDF preview
 7. Zoom in/out for document preview
 8. Page navigation for PDFs
 9. Extraction Trigger
 10. Button to initiate extraction process
 11. Loading/progress indicator while processing
 12. Result Display
 13. Extracted fields should be displayed in:
 - Key-value table format
 - JSON viewer format
 14. JSON Download
 15. Allow user to download extracted JSON response
 16. Error Handling
 17. Invalid file format validation
 18. File size validation
 19. Proper error messages for OCR failure
 20. Responsive UI
 21. Desktop compatible
 22. Mobile responsive design
-

3. Backend Requirements (Python)

The backend should be developed using Python.

Recommended frameworks:

- FastAPI (Preferred) OR
 - Flask
-

3.1 Backend Features

File Handling

Backend should:

- Accept uploaded images and PDFs
- Store temporary files securely
- Support multi-page PDF processing

OCR & Text Extraction

Backend should:

- Read printed text
- Read handwritten text
- Support noisy/scanned documents
- Support low-quality images
- Detect form fields and values

JSON Conversion

System should map extracted text into structured JSON format.

Example:

```
{
  "name": "Riya Sharma",
  "address": "Aleo Post Office Old Manali",
  "city": "Manali",
  "state": "Himachal Pradesh",
  "occupation": "Tour Guide",
  "email": "riyasharma@gmail.com"
}
```

API Requirements

Upload API

`POST /upload`

Request:

- Multipart form-data
- File upload

Response:

```
{
  "status": "success",
  "document_id": "12345"
}
```

Extraction API

`POST /extract`

Response:

```
{
  "status": "success",
  "data": {
    "name": "Riya Sharma",
    "city": "Manali"
  }
}
```

4. AI / OCR Requirements

Important Restriction

LLM usage is NOT allowed.

The system must NOT use:

- OpenAI APIs
- Claude

- Gemini
 - Any cloud-hosted LLM service
 - Any external AI API
-

Allowed Technologies

Small Language Models (SLMs) are allowed only if:

- They run completely locally
 - No internet dependency exists
 - No external API calls are made
 - Data remains within local infrastructure
-

5. Recommended OCR / AI Technologies

Computer Vision Requirements

Computer vision techniques should be used for:

- Form alignment
 - Noise removal
 - Skew correction
 - Region detection
 - Field-value mapping
 - Bounding box detection
 - Table/form structure understanding
-

6. Data Extraction Logic

The system should:

1. Detect document layout
2. Identify labels/fields
3. Detect associated values
4. Handle handwritten entries
5. Normalize extracted values
6. Return structured JSON output

Example Mapping:

Form Label	JSON Key
Name of Proposer	name
City	city

Form Label	JSON Key
State	state
Occupation	occupation
E-Mail	email

7. Non-Functional Requirements

Security

- No cloud processing
- Local-only execution
- Uploaded files should not be permanently stored
- Secure temporary storage cleanup

Performance

- Extraction should complete within acceptable response time
- Support large PDFs
- Concurrent request handling support

Scalability

- Modular architecture
 - Separate OCR and extraction services
 - API-first architecture
-

8. Suggested Architecture

Frontend

- React
- Axios
- Material UI / Tailwind CSS
- React Dropzone

Backend

- Python
- FastAPI
- Uvicorn

OCR & CV

- PaddleOCR
- OpenCV
- Tesseract
- LayoutParser

Storage

- Temporary local storage
 - Optional database for logs/history
-

9. Workflow

1. User uploads image/PDF from React UI
 2. Backend receives document
 3. Preprocessing is applied using OpenCV
 4. OCR extracts printed + handwritten text
 5. Field-value pairs are detected
 6. Structured JSON is generated
 7. JSON response is returned to frontend
 8. Frontend displays extracted data
-

10. Future Enhancements

- Template-based extraction
 - Multi-language support
 - Confidence score display
 - Human review interface
 - Training custom local handwritten OCR model
 - Batch document processing
 - Export to Excel/CSV
-

11. Deliverables

The project deliverables should include:

1. React frontend application
2. Python backend APIs
3. OCR extraction pipeline
4. Local AI/SLM integration
5. JSON extraction engine
6. API documentation
7. Deployment guide

12. Sample Expected Output

```
{  
  "name": "Riya Sharma",  
  "city": "Manali",  
  "state": "Himachal Pradesh",  
  "occupation": "Tour Guide",  
  "email": "riyasharma@gmail.com",  
  "pin_code": "175131"  
}
```